

Pemrograman Berorientasi Objek



MVC

Outline

- ▶ MVC Concept
- ▶ MVC Implementation: Web App
 - Web Application Architecture
 - Web Technologies

About MVC [1]

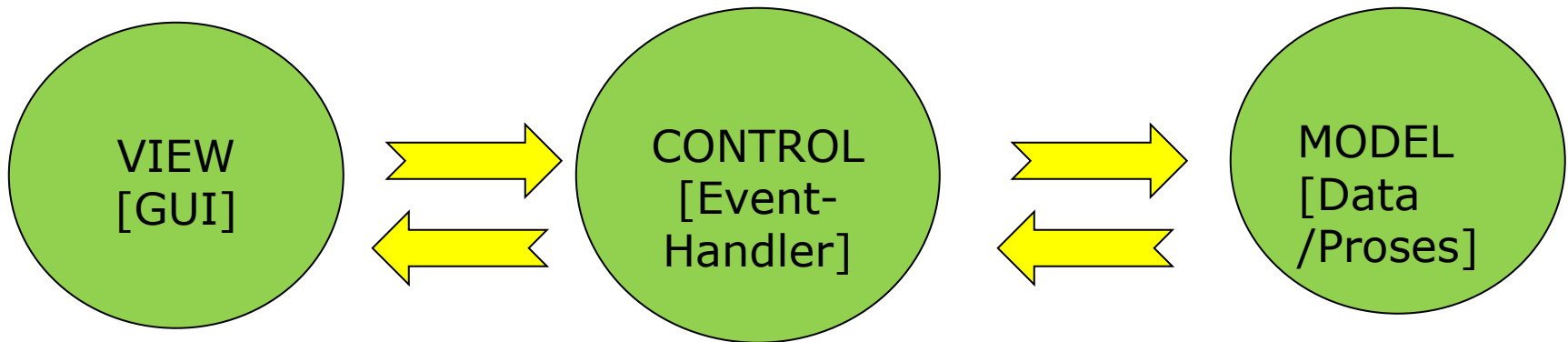
- ▶ The model-view-controller (MVC) architecture provides a set of design patterns that help you separate the areas of concern involved in building and running a GUI-based application.
- ▶ The *model encapsulates the business logic and persistence code for the application*. The model should be as view-technology-agnostic as possible.

About MVC [2]

- ▶ The *view should display model objects and contain presentation logic* only. There should be no business logic or controller logic in the view
- ▶ The *controller (along with its attending logic) acts as the mediator* between the view and the model. The controller talks to the model and delivers model objects to the view to display.

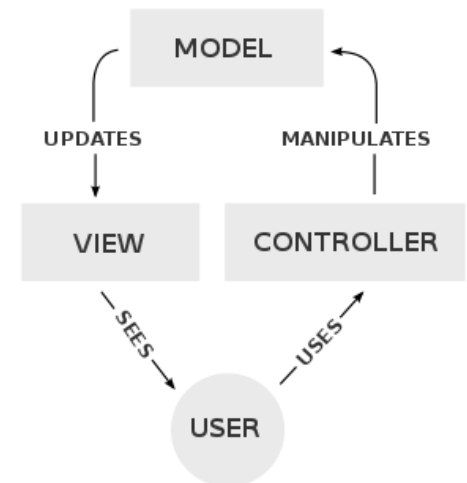
Architecture vs Pattern ?

- ▶ Define Clear Responsibilities, communication and loose coupling between the three primary components of most user systems



Note

- MVC is a software architectural pattern
- This separation of roles allows software development, testing, and maintenance for each role to be more flexible and independent



Model

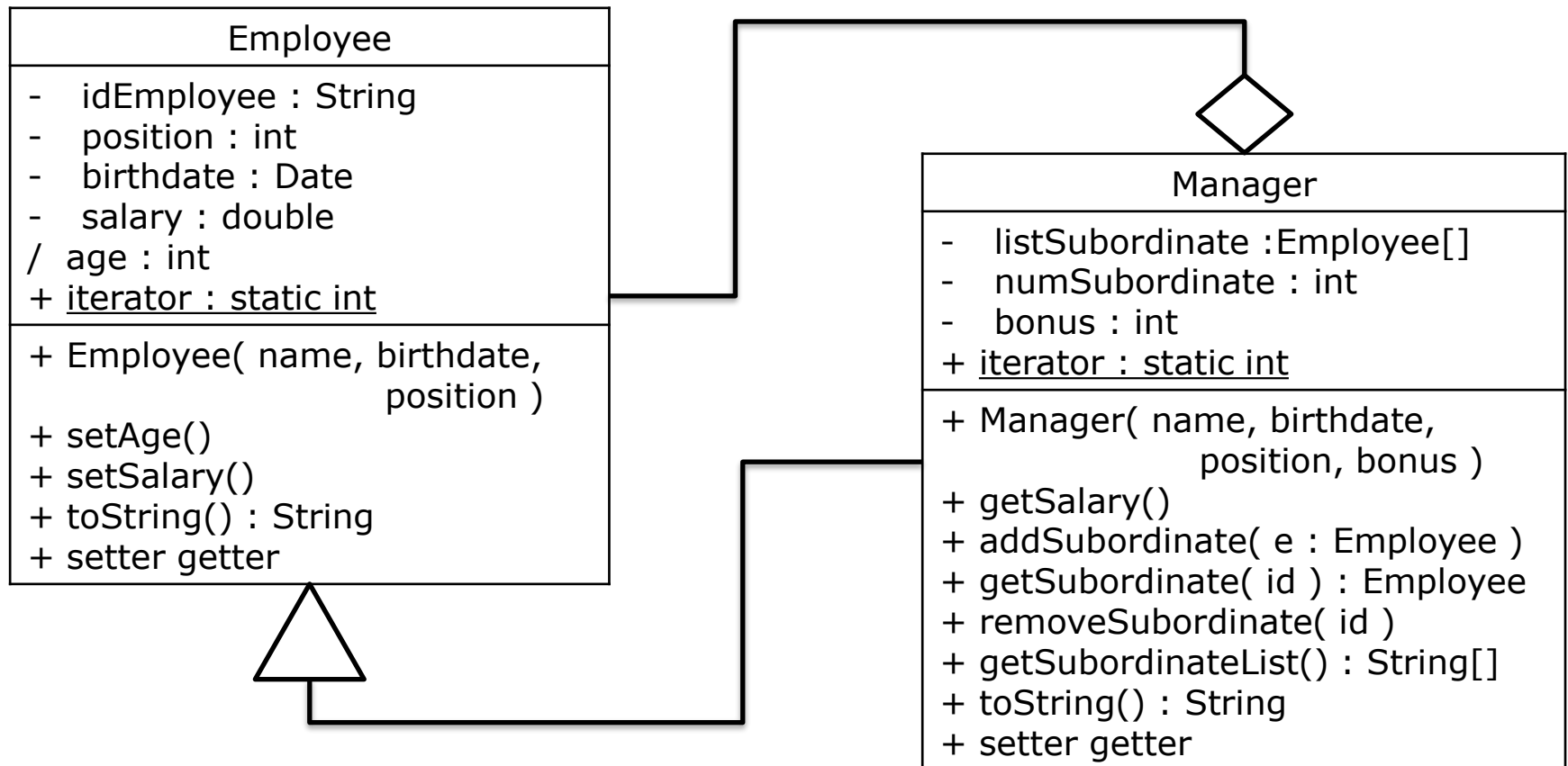
- ▶ Manage the data behavior of application scope
- ▶ Responds request from Controller to do data management process
- ▶ Example: class that represents database table, active record

Model Class

- ▶ In Java, also called Java Bean
- ▶ Simple Java class
 - also called POJO (Plain Old Java Object)
- ▶ Class that represents entity or database table



POJO Example : Employee - Manager



Employee class

```
public class Employee implements Serializable {  
  
    private String idEmployee;  
    private String name;  
    private int position;  
    private Date birthDate;  
    private double salary;  
  
    private transient int age;  
    public static int iterator;  
  
    public Employee(String name, Date birthDate, int position) {  
        this.name = name;  
        this.birthDate = birthDate;  
        iterator++;  
        this.idEmployee = "PG-" + (iterator);  
        this.position = position;  
        setAge();  
        setSalary();  
    }  
}
```

```
    public void setAge() {  
        age = (new Date()).getYear() - birthDate.getYear();  
    }  
  
    public void setSalary() {  
        switch (position) {  
            case 1:  
                salary = 10000;  
                break;  
            case 2:  
                salary = 20000;  
                break;  
            case 3:  
                salary = 30000;  
                break;  
        }  
    }  
  
    @Override  
    public String toString() {  
        return "\nEmployee ID=" + idEmployee  
            + "\nName=" + name  
            + "\nBirthDate=" + birthDate.toLocaleString()  
            + "\nAge=" + age  
            + "\nposition=" + position  
            + "\nsalary=" + salary;  
    }  
}
```

Manager class

```
public class Manager extends Employee implements Serializable {  
  
    private Employee listSubordinate[];  
    private int numSubordinate;  
    private int bonus;  
    public static int iterator = 0;  
  
    public Manager(String name, Date birthDate, int position, int bonus) {  
        super(name, birthDate, position);  
        iterator++;  
        setIdEmployee("MG-" + (iterator));  
        this.bonus = bonus;  
        listSubordinate = new Employee[10];  
        numSubordinate = 0;  
    }  
  
    @Override  
    public double getSalary() {  
        return super.getSalary() + bonus;  
    }  
  
    public Employee getSubordinate(String id) {  
        for (int i = 0; i < getNumSubordinate(); i++) {  
            if (listSubordinate[i].getIdEmployee().equals(id)) {  
                return listSubordinate[i];  
            }  
        }  
        return null;  
    }  
}
```

```
    public void addSubordinate(Employee e) {  
        if (numSubordinate < 10) {  
            this.listSubordinate[numSubordinate++] = e;  
        } else {  
            throw new IllegalArgumentException("Manager is already full");  
        }  
    }  
  
    public void removeSubordinate(String id) {  
        for (int i = 0; i < getNumSubordinate(); i++) {  
            if (listSubordinate[i].getIdEmployee().equals(id)) {  
                for (int j = i; j < getNumSubordinate(); j++) {  
                    listSubordinate[j] = listSubordinate[j + 1];  
                }  
            }  
            listSubordinate[numSubordinate--] = null;  
        }  
    }  
  
    public String[] getSubordinateList() {  
        String a[] = new String[numSubordinate];  
        for (int i = 0; i < numSubordinate; i++) {  
            a[i] = listSubordinate[i].getIdEmployee();  
        }  
        return a;  
    }  
}
```

View

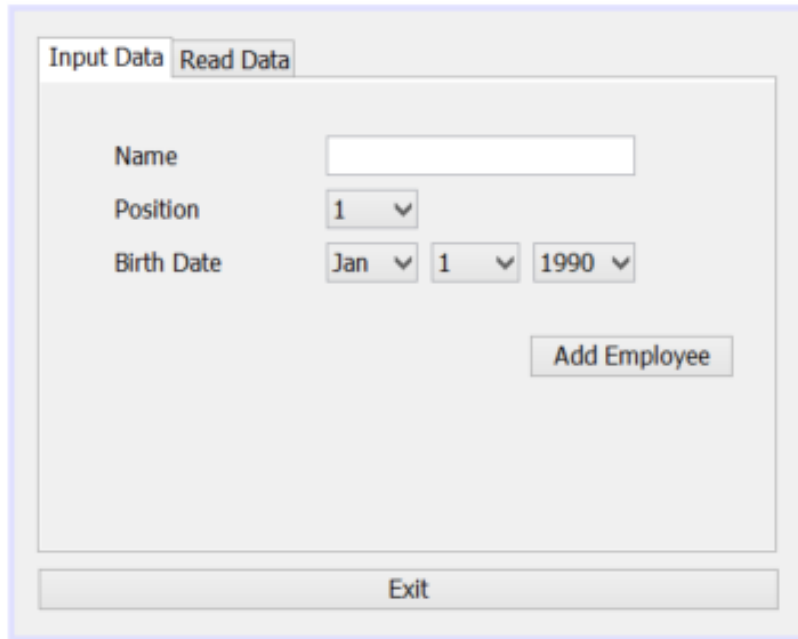
- ▶ The UI or forms to interact with user
- ▶ One Controller can use many Views for different needs
- ▶ Example: login page, registration form, dashboard page

View Class

- ▶ Purely to receive input and display output
- ▶ No business or logic process
 - Unless it is closely related to the display procedure (presentation logic)
- ▶ View class usually exists in desktop app. For web and mobile app, most common practice is there's no view class (the view is a interpreted file, not a Java class)



View Class: Container



Input Data Read Data

Name

Position 1 ▾

Birth Date Jan ▾ 1 ▾ 1990 ▾

Add Employee

Exit

View Class: Listener

- Define the business process when the button pressed in actionPerformed method

```
public void actionPerformed(ActionEvent ae) {  
    Object source = ae.getSource();  
    if (source.equals(btnExit)) {  
        System.exit(0);  
    } else if (source.equals(btnLoad)) {  
  
        // what to do when button is pressed  
  
    } else if (source.equals(btnAdd)) {  
  
        // what to do when button is pressed  
  
    } else if (source.equals(btnAge)) {  
  
        // what to do when button is pressed  
    }  
}
```

View File

- ▶ For web and mobile app, the view is an interpreted text file
 - Container and layout: HTML
 - Styling: CSS
 - Listener: Javascript
- ▶ We will use this in this course. See the practical work module for details. Explore it yourself

Controller

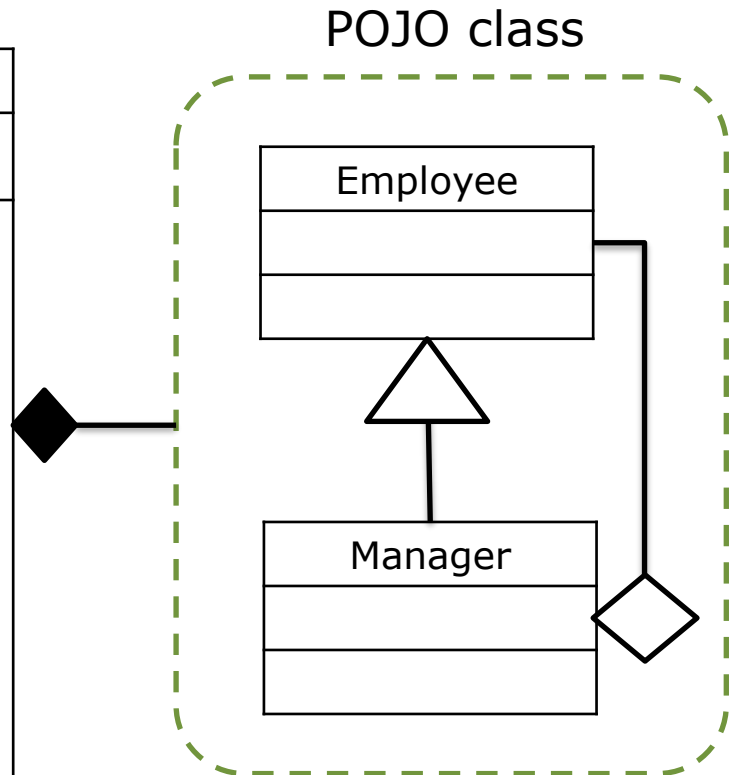
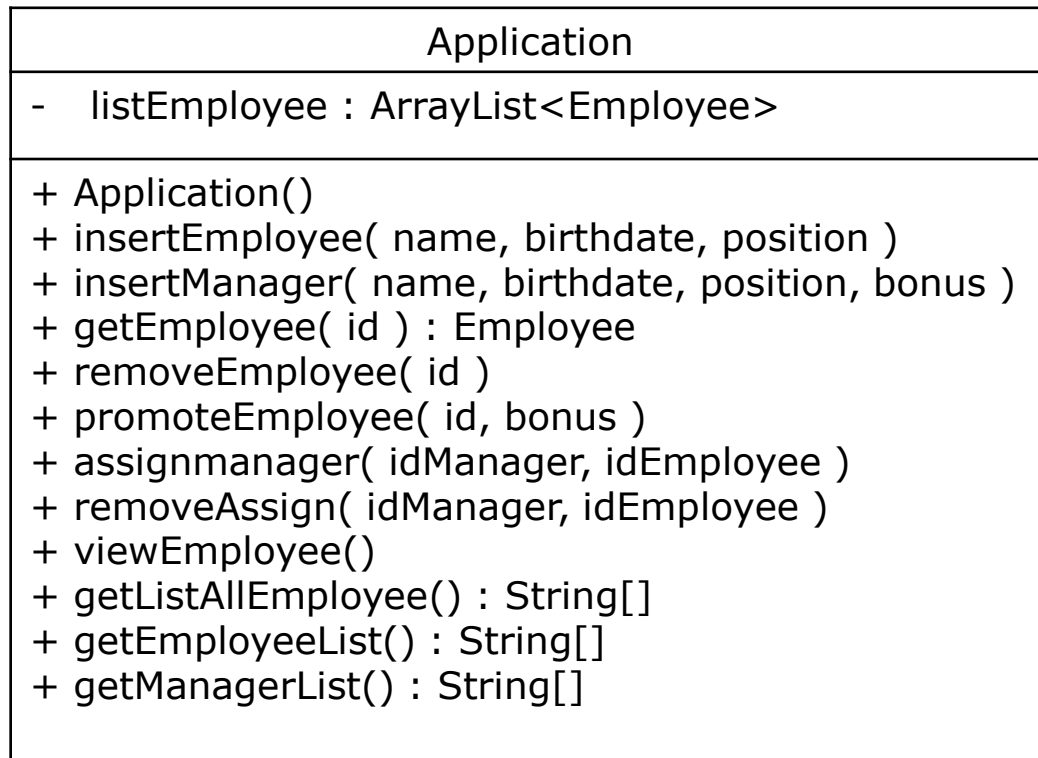
- ▶ Accepts and process the user inputs from View
- ▶ Responds with Model or View calling
- ▶ Give instructions to Model / View to do some action
- ▶ Example: Class that isn't a Model or a View, usually contains logic and business process

Controller Class

- ▶ Contains business process functions
 - as defined in use case diagram
- ▶ Where the other object lives
- ▶ For Java in web, it's also called Servlet



Controller Class



Controller Class

```
public class Application {  
  
    private ArrayList<Employee> listEmployee;  
  
    public Application() {  
        listEmployee = new ArrayList<>();  
    }  
  
    public void insertEmployee(String name, Date birthDate, int position) {  
        listEmployee.add(new Employee(name, birthDate, position));  
    }  
  
    public void insertManager(String name, Date birthDate, int position, int bonus) {  
        listEmployee.add(new Manager(name, birthDate, position, bonus));  
    }  
  
    public Employee getEmployee(String id) {  
        for (Employee daftarP1 : listEmployee) {  
            if (daftarP1.getIdEmployee().equals(id)) {  
                return daftarP1;  
            }  
        }  
        return null;  
    }  
}
```

Controller Class

```
public void removeEmployee(String id) {
    for (int i = 0; i < listEmployee.size(); i++) {
        Employee e = listEmployee.get(i);
        if (e.getIdEmployee().equals(id)) {
            if (e instanceof Manager && ((Manager) e).getNumSubordinate() != 0) {
                throw new IllegalArgumentException("Manager still has subordinate(s)");
            }
            for (Employee e2 : listEmployee) {
                if (e2 instanceof Manager && ((Manager) e2).getSubordinate(e.getIdEmployee()) != null) {
                    throw new IllegalArgumentException("Employee is still assigned to a Manager");
                }
            }
            listEmployee.remove(i);
        }
    }
}

public void promoteEmployee(String id, int bonus) {
    Employee e = getEmployee(id);
    if (e != null) {
        if (!(e instanceof Manager)) {
            removeEmployee(id);
            listEmployee.add(new Manager(e.getName(), e.getBirthDate(), e.getPosition(), bonus));
        } else {
            throw new IllegalArgumentException("Employee is already a Manager");
        }
    } else {
        throw new IllegalArgumentException("Employee not found");
    }
}
```

Controller Class

```
public void assignManager(String idM, String idP) {
    Manager m;
    Employee e = getEmployee(idM);
    if ((e != null) && (e instanceof Manager)) {
        m = (Manager) e;
    } else {
        throw new IllegalArgumentException("Manager not found");
    }
    e = getEmployee(idP);
    if ((e != null) && (e instanceof Employee)) {
        if (m.getSubordinate(e.getIdEmployee()) == null) {
            m.addSubordinate(e);
        } else {
            throw new IllegalArgumentException("Employee is already in Manager's list");
        }
    } else {
        throw new IllegalArgumentException("Employee not found");
    }
}

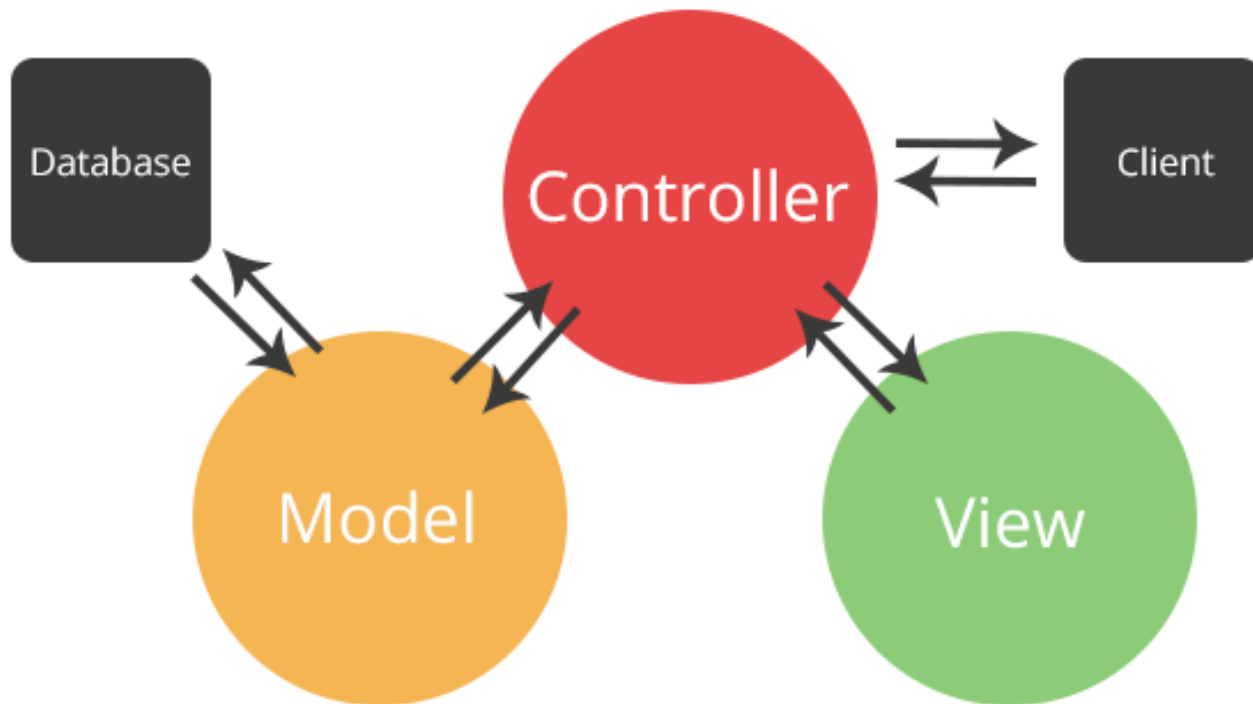
public void removeAssign(String idM, String idP) {
    Manager m;
    Employee e = getEmployee(idM);
    if ((e != null) && (e instanceof Manager)) {
        m = (Manager) e;
    } else {
        throw new IllegalArgumentException("Manager not found");
    }
    m.removeSubordinate(idP);
}
```

Controller Class

```
public void viewEmployee() {  
    for (Employee e : listEmployee) {  
        System.out.println("-----");  
        System.out.println(e.toString());  
    }  
}  
  
public String[] getListAllEmployee() {  
    String a[] = new String[listEmployee.size()];  
    int j = 0;  
    for (Employee e : listEmployee) {  
        a[j++] = e.getIdEmployee();  
    }  
    return a;  
}
```

```
public String[] getEmployeeList() {  
    String a[] = new String[listEmployee.size()];  
    int j = 0;  
    for (Employee e : listEmployee) {  
        if (!(e instanceof Manager)) {  
            a[j++] = e.getIdEmployee();  
        }  
    }  
    String b[] = new String[j];  
    System.arraycopy(a, 0, b, 0, j);  
    return b;  
}  
  
public String[] getManagerList() {  
    String a[] = new String[listEmployee.size()];  
    int j = 0;  
    for (Employee e : listEmployee) {  
        if (e instanceof Manager) {  
            a[j++] = e.getIdEmployee();  
        }  
    }  
    String b[] = new String[j];  
    System.arraycopy(a, 0, b, 0, j);  
    return b;  
}
```

MVC Flow



MVC Flow

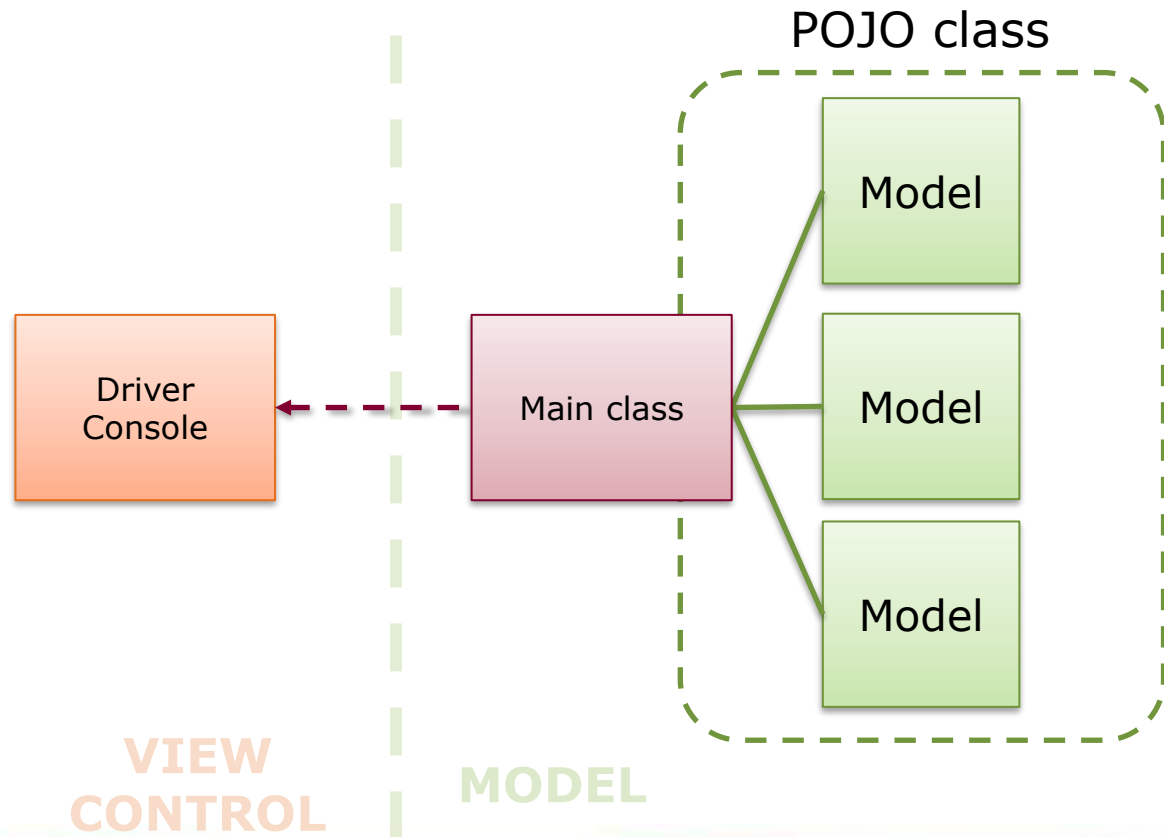
- User open a URL address, Controller that mapped to that address returns the View
- User interacts with View (UI) (example: fill the input field or click a button)
- Controller handles the inputs from View, runs the logic / calls and instructs the Model
- Model runs the instructions
- Model give a return message / data to Controller
- Controller return the message / data to View and/or return another View

Note

- There's a lot of approach to achieve it according to what programming language and coding style of the programmer
 - MVC coding style might differ from one programmer to another
 - The example here is only one of those

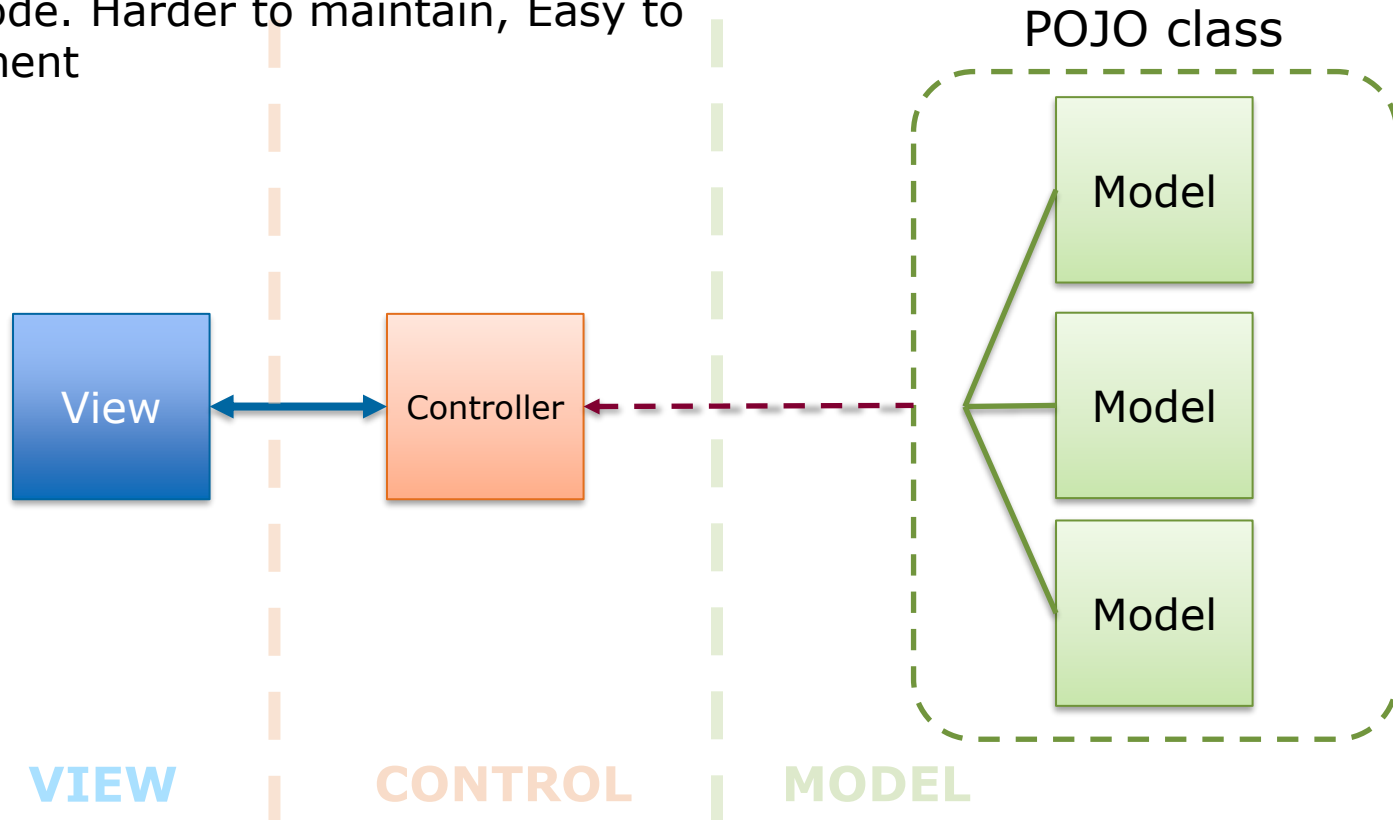


MVC Approach : Driver Console

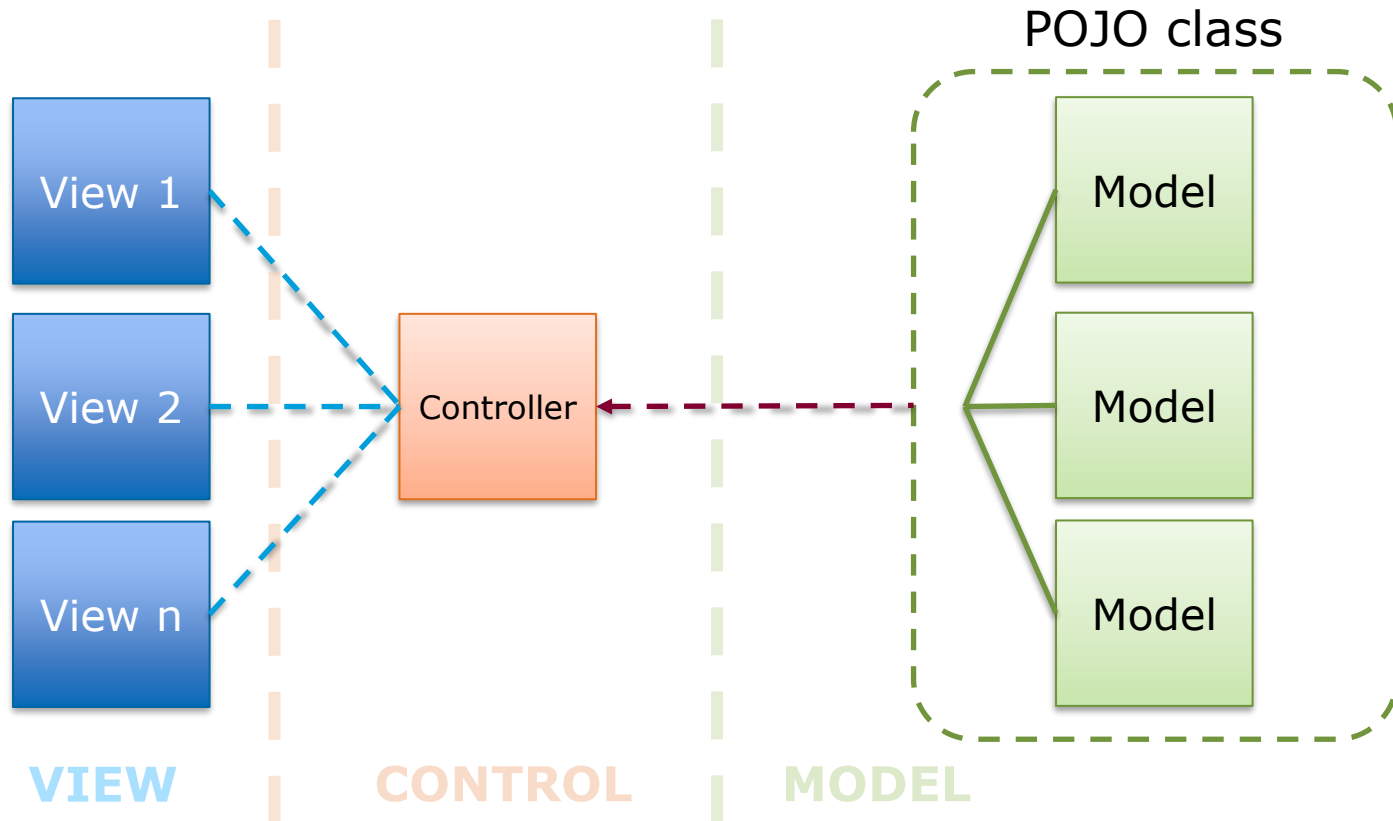


MVC Approach : 1 view - 1 controller

Controller will be a 'fat' class, View will be a long code. Harder to maintain, Easy to implement

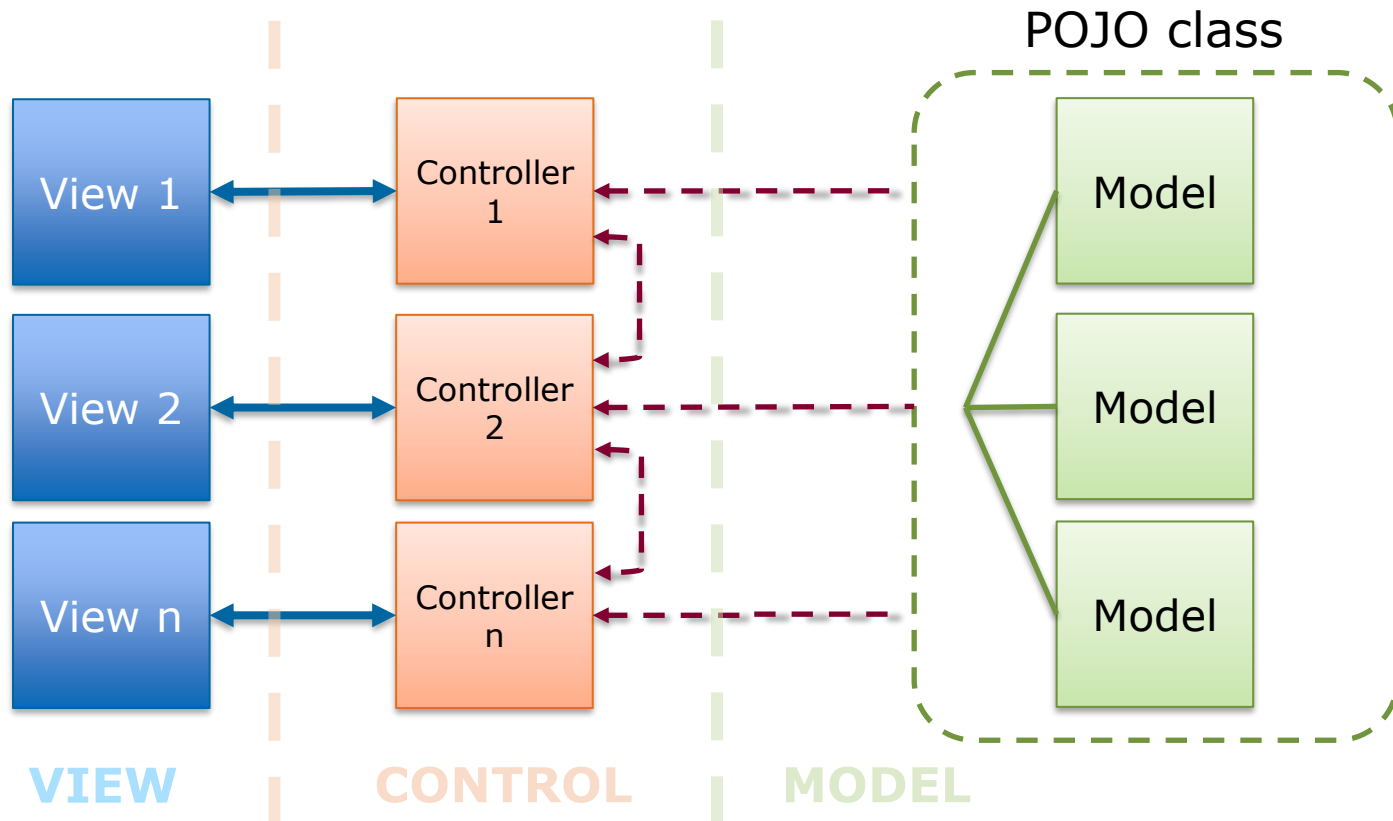


MVC Approach : n view - 1 controller



MVC Approach : n view - n controller

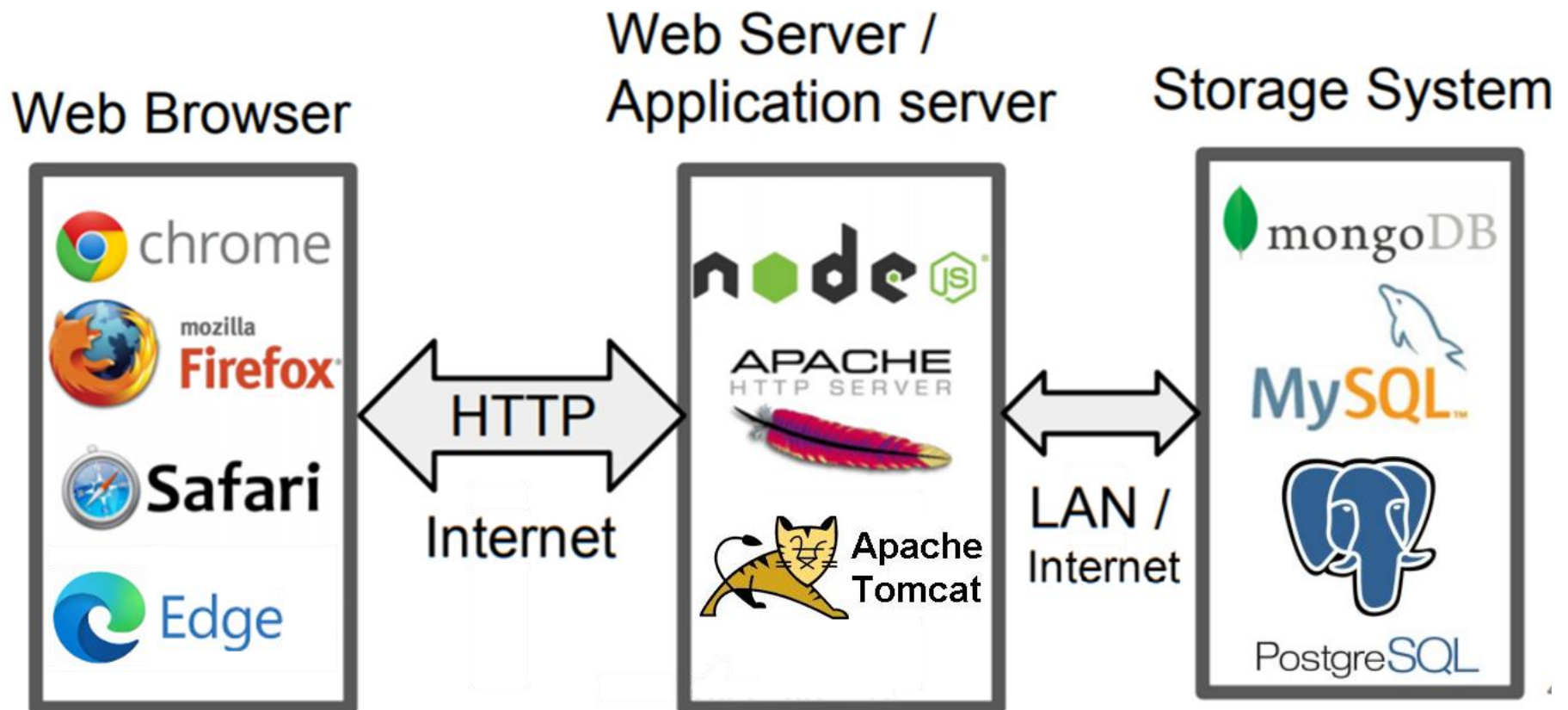
Most projects and frameworks now





MVC Implementation: Java Web

Web Application Architecture



How web server and browser “talk”?

- ▶ They communicate using HyperText Transfer Protocol (HTTP).
- ▶ It is a request-response pattern, meaning a request must be made to receive a response.

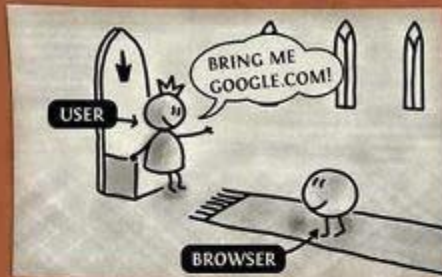


Http

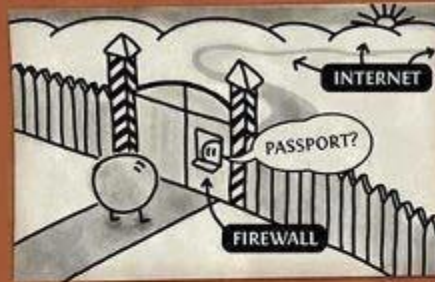


Https

How Web Browser Works..



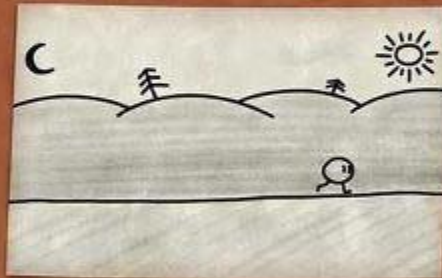
1



2



3



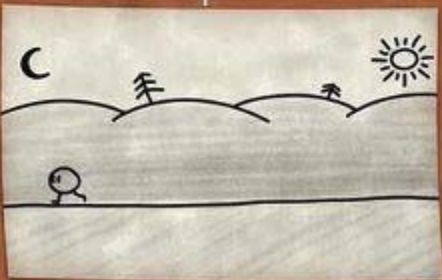
4



5



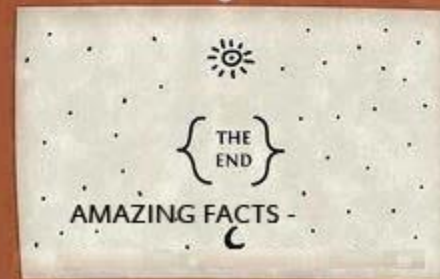
6



7

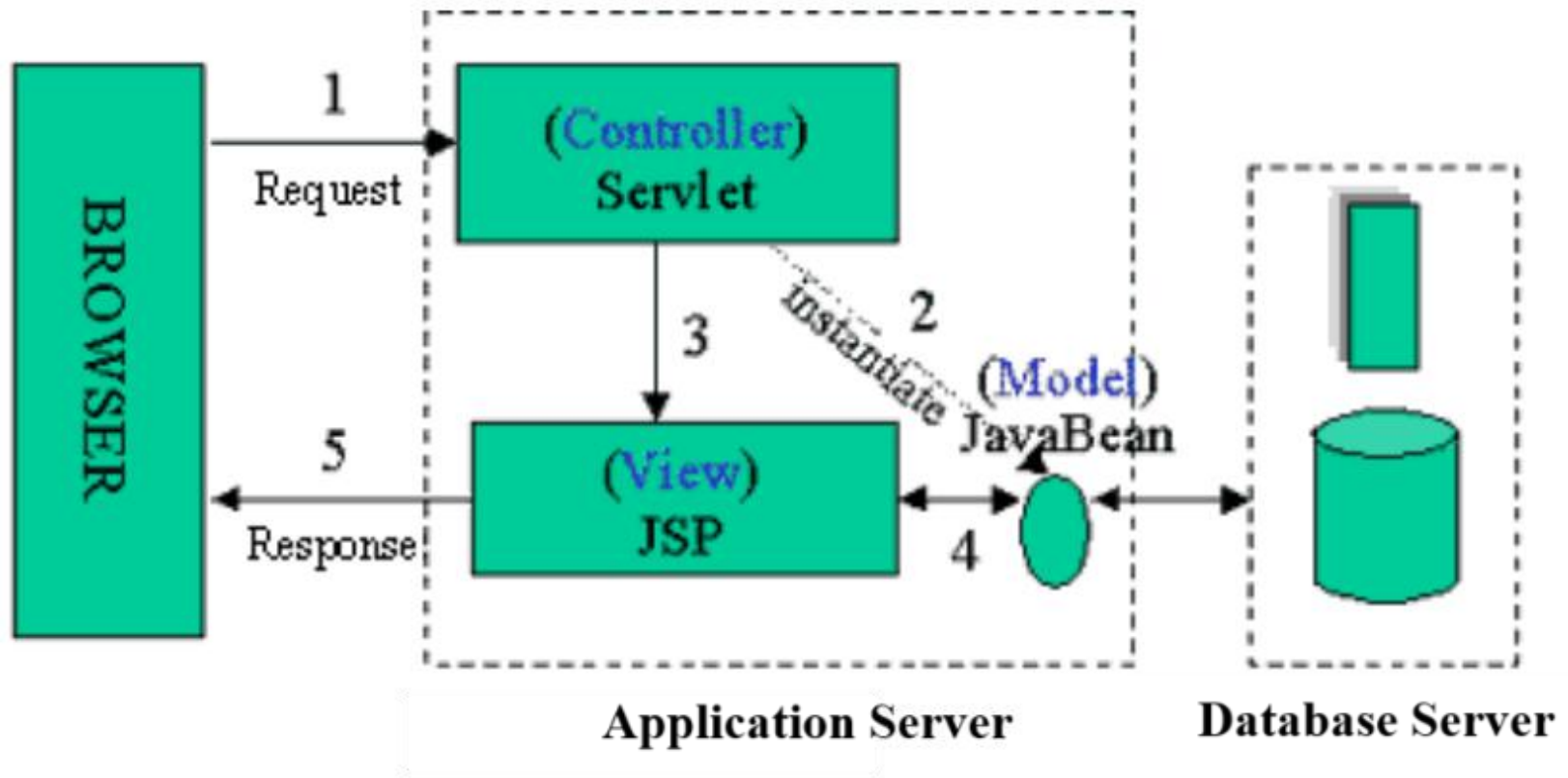


8



9

MVC Architecture in Java Web



Web Technologies

- ▶ Client Side Scripting (frontend)
 - Hypertext Markup Language (HTML)
 - Cascading Style Sheet (CSS)
 - Javascript



HTML
The Skeleton



CSS
The Skin

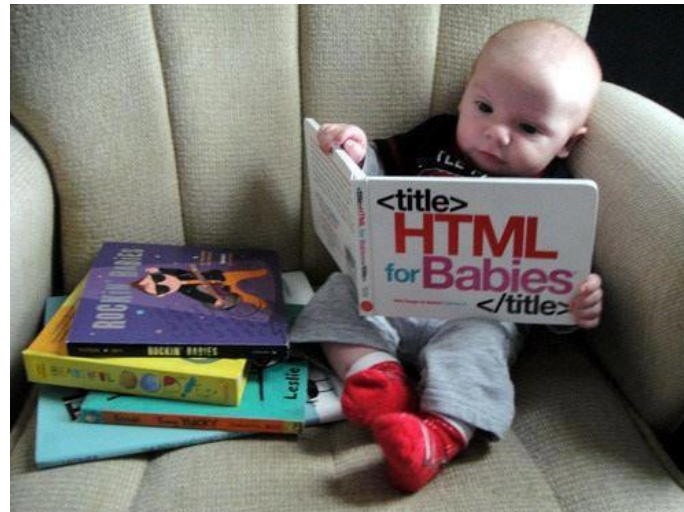


JavaScript
The Brain

- ▶ Server Side Scripting (backend)

HTML

- Describe content and defines the structure within webpage.
- Hypertext documents on the web are designed to be linked to other documents.



HTML

- ▶ Markup Language: set of tags that are added to the text of web page to define its structure



```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>

<h1>Main Headline</h1>
<p>A paragraph.</p>

</body>
</html>
```

CSS

- Design plays an important part and is a key component of every successful website.
- Some people interested into finding the look and feel of a site, and enjoy working on the User Experience/UX.
- Some people prefer to work on the functionality of a site called User Interface/UI.
- All developers need to learn how CSS influences a document.

CSS

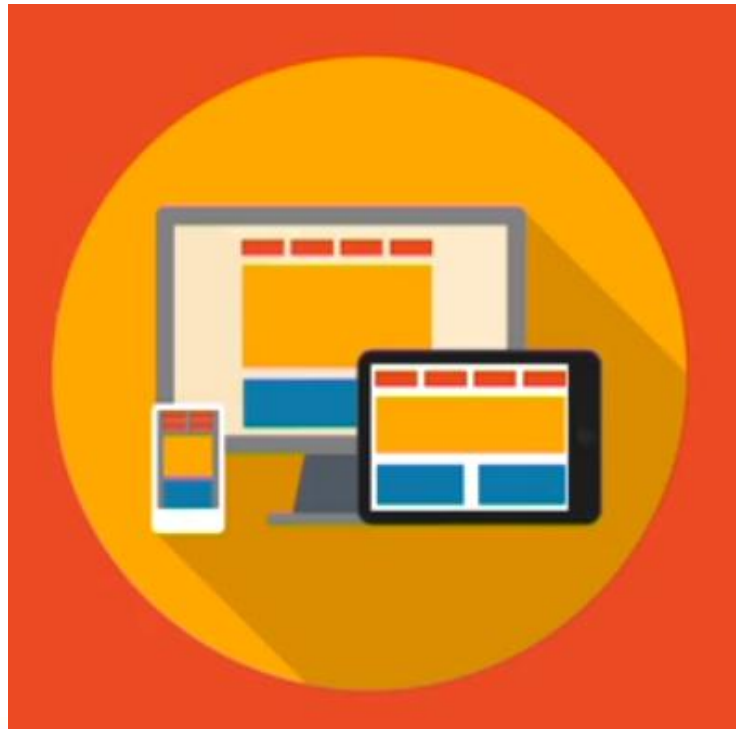
- ▶ Set of rules, or style sheets, for how the style is translated into visual form
- ▶ Cascading style rules priority of how the styles are rendered on a page

CSS



CSS

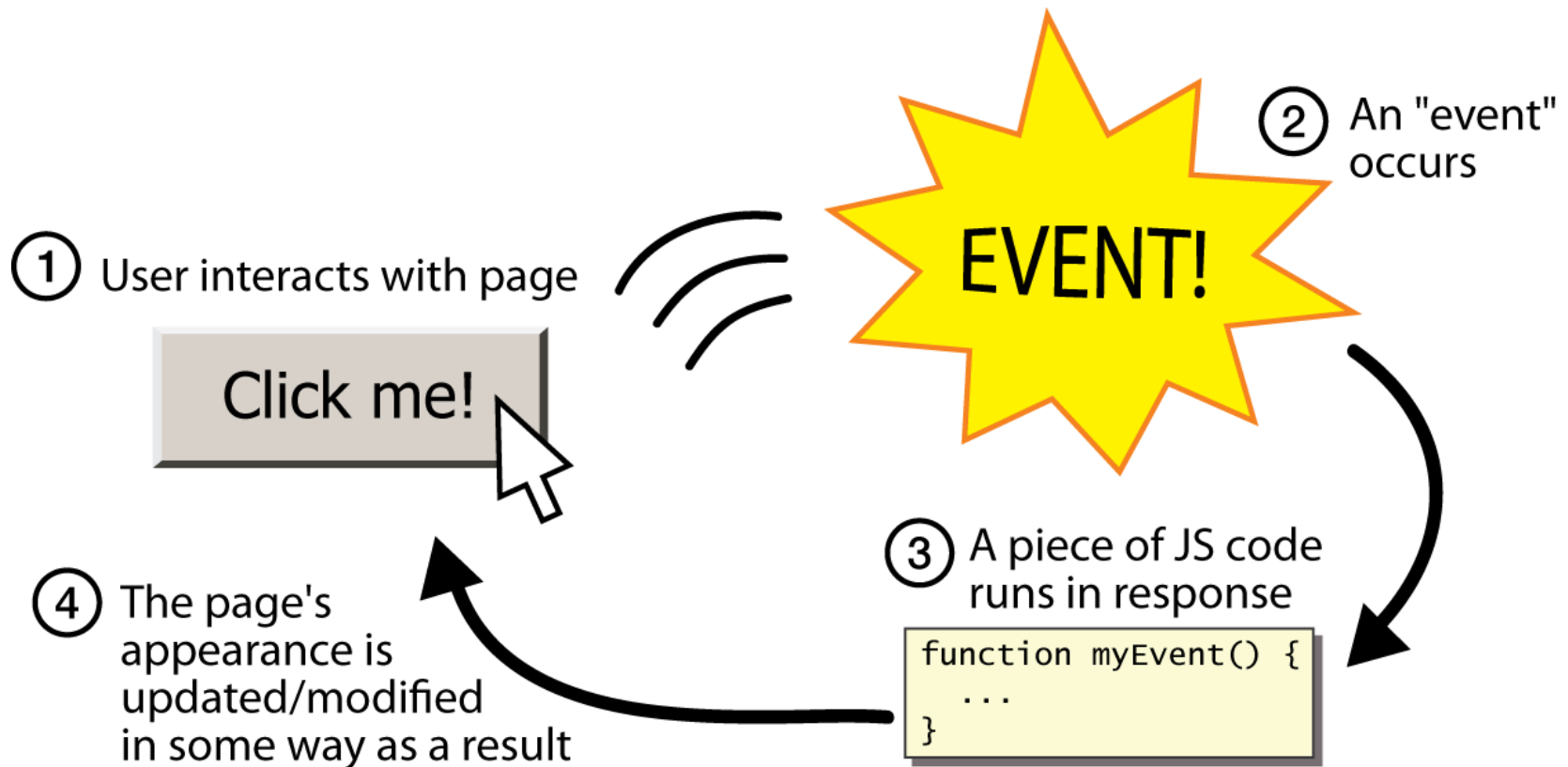
- ▶ Ultimate goal: Design one which compatible to all (Desktop, Mobile, Tablet)



Javascript

- ▶ Language responsible for managing interaction within browser.
- ▶ Handling events, loading and unloading media, and creating modifying and deleting elements in a page
- ▶ Tools : jQuery, DataTables, Select2

JS is Event-driven programming



Why We Need Front-end Framework

- ▶ Build a layout from scratch with HTML & CSS takes a lot of work and code
- ▶ We want to develop front-end faster, so the back-end coder can code the task earlier
- ▶ We build the web app for many platforms

Advantages?

- ▶ Makes life easier for developers 😊. Front-end framework greatly saves development time.
- ▶ Front-end framework usually has cutting edge interface.
- ▶ Mobile Friendly, supports all device resolution (responsive).
- ▶ Compatible with all modern browsers.

Disadvantages?

- ▶ Lots of codes.
- ▶ Many websites use it. So it could happen that your website appearance is similar to others.
- ▶ Some frameworks takes more development time because they require another environment to build the layouts and to run it

Front-end Frameworks Today

- ▶ Bootstrap
- ▶ Foundation
- ▶ TailwindCSS
- ▶ Angular
- ▶ React / Next
- ▶ Vue / Nuxt
- ▶ Svelte
- ▶ JokoUI 🤔 🤔 🚫 🚫

Server Side Scripting

- ▶ Massive amounts of information makes it impractical to custom-build every single page using only frontend: HTML, CSS, and Javascript.
- ▶ We use programming language for data and logical processing in backend
- ▶ Most backends using database to manage the flow of information sent to the user

Programming Language for Web

- ▶ PHP
- ▶ Java
- ▶ NodeJS
- ▶ C# / Basic (ASP .NET)
- ▶ Ruby
- ▶ Python
- ▶ Go

In this course, we will use:

- ▶ Java
 - You have learnt it, so it's "easier"
 - Robust, Secure, Fast

DBMS Type

▶ Relational

- Organize data into tables, where data is structured in rows and columns

▶ Non-Relational

- Stores data using document, graph, or key-value formats

DBMS Examples



Need more corporate and power? You can try



Notes in Server Side Scripting: Database

- ▶ How to access your data in database
 - Connection: local server (LAN) or remote server (GCP, AWS)
 - Query: SQL native or using Object Relational Model (Hibernate, EJB)
- ▶ DBMS uses SQL and other SQL inheritance
 - Some DBMS has their own additional self-made query
- ▶ Most common is using Relational DB
 - MySQL / MariaDB, PostgreSQL, Oracle, SQL Server
- ▶ Using Non-Relational DB is good too, but will make the implementation of DB schema more complex
 - MongoDB, Google Firebase / Firestore

Notes in Server Side Scripting: Designing URLs for Readability

Using **Nouns** to Represent Resources

- /getProducts ✗
- /products ✓

Avoid adding **file extensions** in URLs

- products.json ✗
- products ✓

Using **Plural** for Resource Collection

- /product ✗
- /products ✓

Using **Hyphens** to Separate Words

- /checkoutorder ✗
- /checkout-order ✓

Front-end vs Back-end

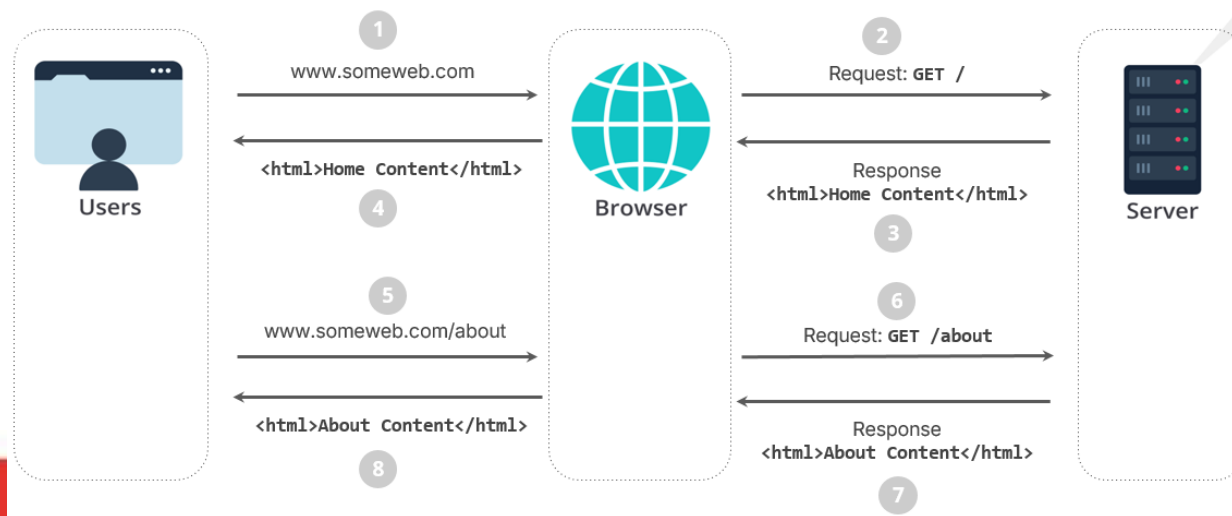
- ▶ Front-end developers build everything that you see (user interface of web and apps).
- ▶ Back-end developers build the part that you don't see (server-side of web and apps).



How to build Front-end and Back-end

► Server-Side Rendering

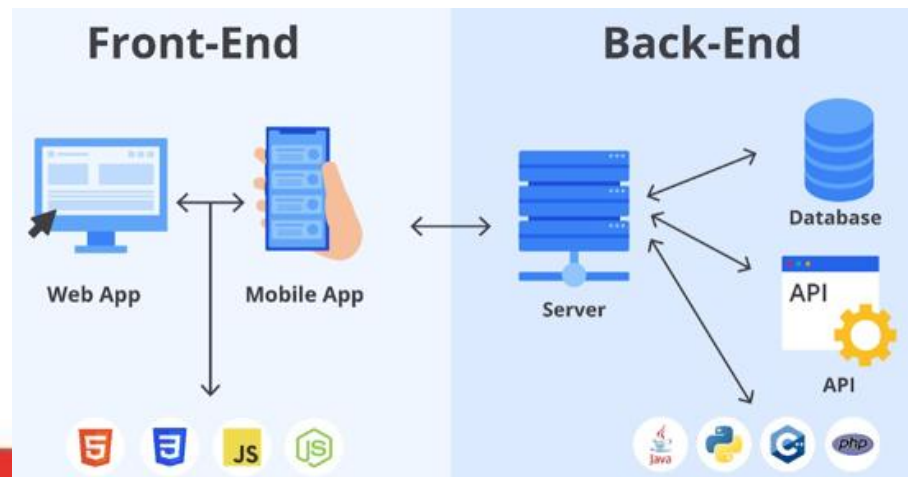
- Backend loads a frontend file, fills it with data from backend (optional, if any), generates the file into a HTML page, and returns the page to browser
- Usually used in monolithic development, where backend and frontend codes are in the same repository/codebase



How to build Front-end and Back-end

► Client-Side Rendering

- Backend doesn't return a page, instead returns a formatted data to browser
- Usually used in API-driven or microservices development (frontend and backend codes are in separate/different repository/codebase)



Application Server for Java Web

- ▶ Apache Tomcat, Jetty
 - lighter, simple
 - Apache Tomcat is included in XAMPP
- ▶ WebLogic, WebSphere, WildFly, Glassfish
 - heavier, more features

Explore more

› HTML

https://telkomuniversityofficial-my.sharepoint.com/:b:/g/personal/monterico_telkomuniversity_ac_id/IQAfWzLh9P4pT4VMZOATVAhfAYR9hmTmMJDcT2p0jWV9Qwk?e=U9WIO5

› CSS & Bootstrap

https://telkomuniversityofficial-my.sharepoint.com/:b:/g/personal/monterico_telkomuniversity_ac_id/IQDSRWq--1mxS73940q40xgSAf6ASPMYFFzvUNFo-778lkc?e=5MumtF

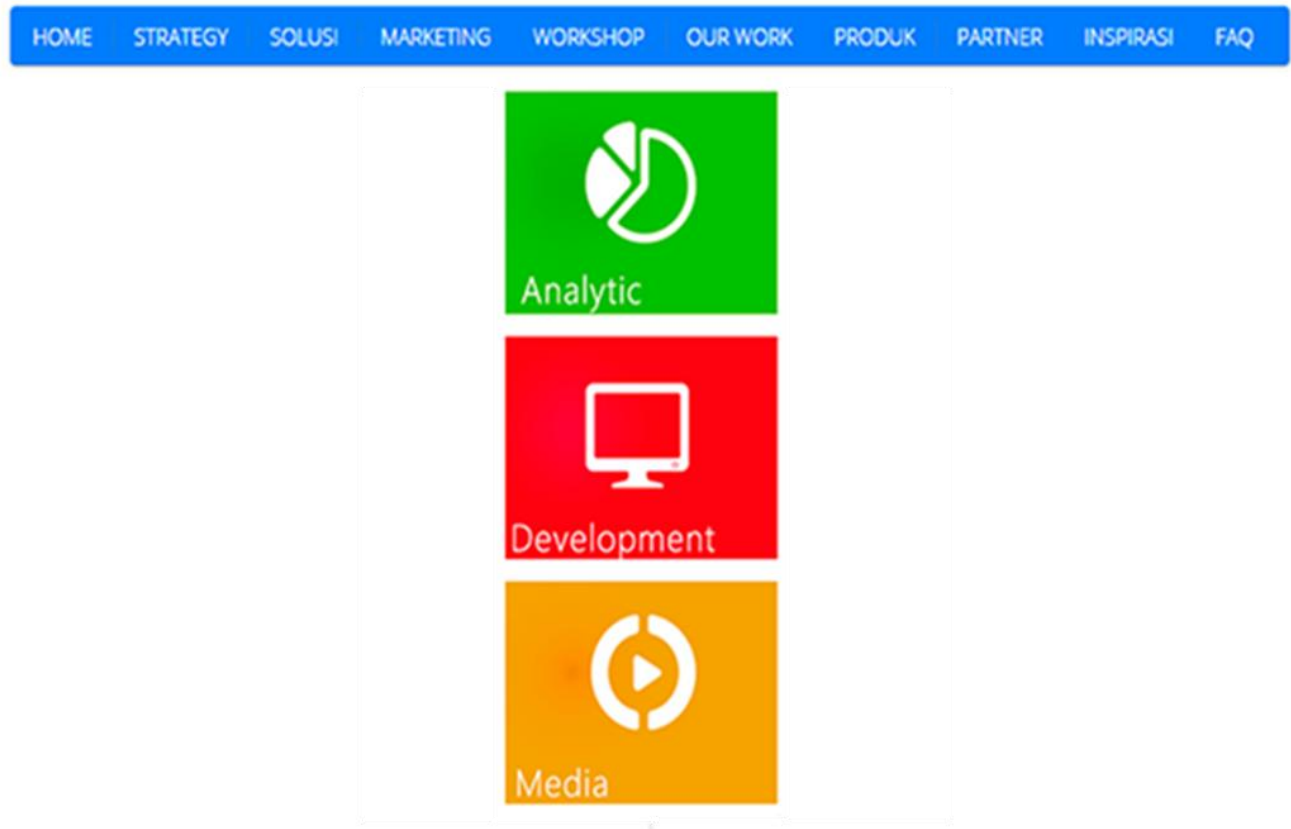
› Javascript & jQuery

https://telkomuniversityofficial-my.sharepoint.com/:b:/g/personal/monterico_telkomuniversity_ac_id/IQDtkHTBxN-3T63-EbFi4UCPAbB62T2hcJJZXpboxFpsRvg?e=rMUc9H

Question?

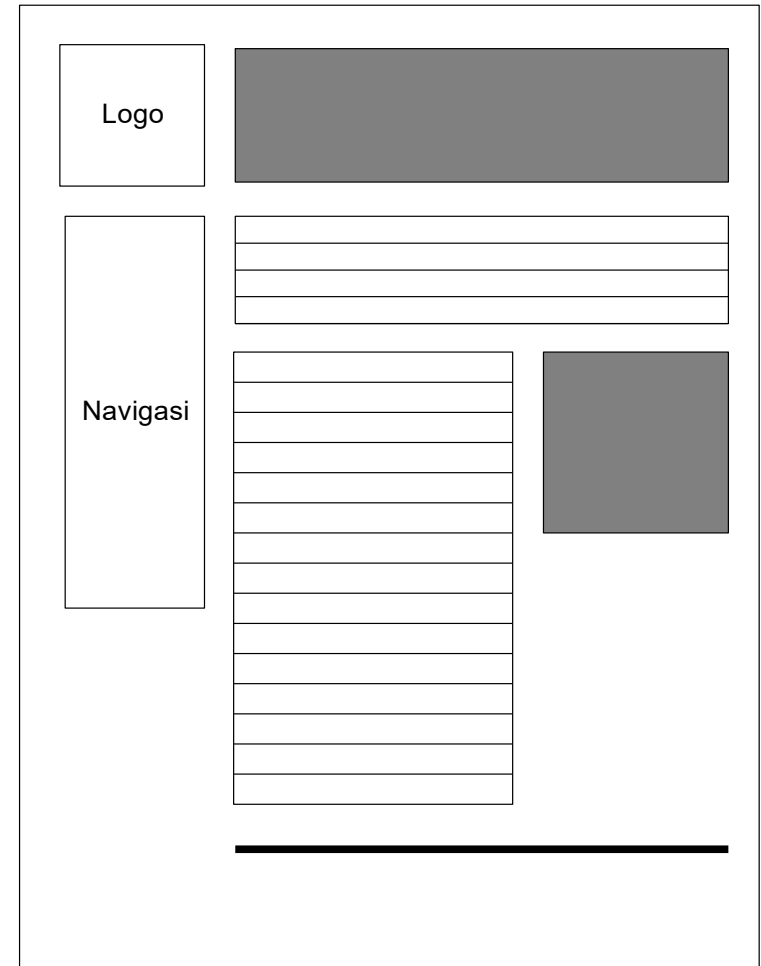


Exercise 1 **ONLINE MEDIA**



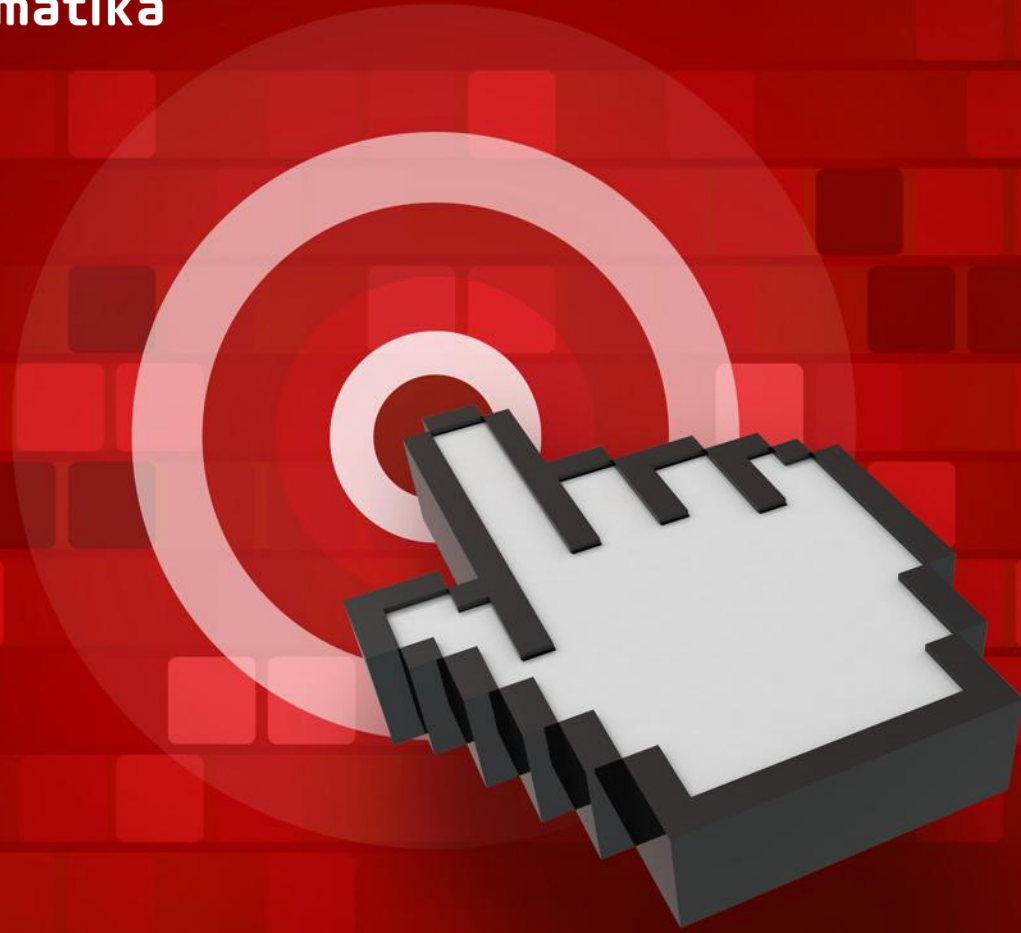
[illegible]

Exercise 3





Fakultas Informatika
School of Computing
Telkom University



THANK YOU